Iteration A2: Making Prettier Listings

Our customer has one more request. (Customers always seem to have one more request, don't they?) The listing of all the products is ugly. Can we pretty it up a bit? And while we're in there, can we also display the product image along with the image URL?

We're faced with a dilemma here. As developers, we're trained to respond to these kinds of requests with a sharp intake of breath, a knowing shake of the head, and a murmured, "You want what?" At the same time, we also like to show off a bit. In the end, the fact that it's fun to make these kinds of changes using Rails wins out, and we fire up our trusty editor.

Before we get too far, though, it would be nice if we had a consistent set of test data to work with. We *could* use our scaffold-generated interface and type data in from the browser. However, if we did this, future developers working on our codebase would have to do the same. And if we were working as part of a team on this project, each member of the team would have to enter his or her own data. It would be nice if we could load the data into our table in a more controlled way. It turns out that we can. Rails has the ability to import seed data.

To start, we modify the file in the `db` directory named `seeds.rb`.

In this file, we add the code to populate the `products` table. This uses the `create!` method of the `Product` model. The following is an extract from that file. Rather than type the file by hand, you might want to download the file from the sample code available online.[38] While you're there, copy the images into the `app/assets/images` directory in your application.[39]

Be warned: this `seeds.rb` script removes existing data from the `products` table before loading the new data. You might not want to run it if you've just spent several hours typing your own data into your application!

rails7/depot_a/db/seeds.rb